CODYSSEY 미션 3 · AI 원리 쉽게 알아보기

AI는 어떻게 모양을 알아볼까?

직접 만든 작은 AI 계산기 **Mini NPU**로 알아보기



코드 작성: AI Claude의 도움을 받아 작성

오늘의 순서

오늘은 4가지 질문에 답해 볼 거예요

01

컴퓨터는 그림을 어떻게 볼까?

개념 · 숫자, 필터, MAC

02

코드로는 어떻게 만들까?

코드 · main.py 핵심 함수

03

정답인데 왜 FAIL이 나올까?

오차 · 컴퓨터의 계산 실수

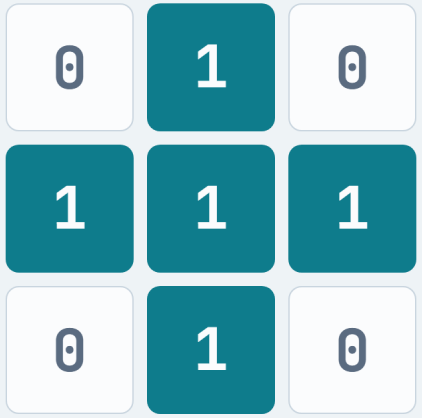
04

더 크게, 더 많이 하려면?

속도 · 고장 찾기 · 확장

코드를 몰라도 괜찮아요. 모든 코드는 **우리말로 한 줄씩** 풀어서 설명할게요.

컴퓨터 눈에는 그림이 아니라 숫자만 보여요



칠한 칸 = 1, 빈 칸 = 0
이렇게 숫자를 가로·세로로 늘어놓은 표를 **2차원 배열**이라고 해요.

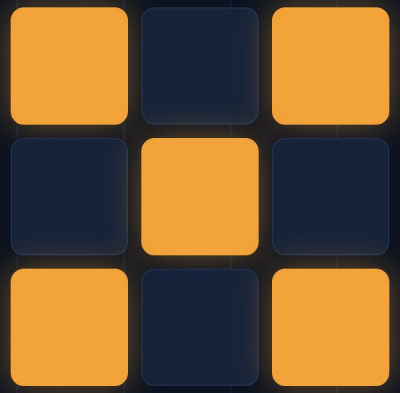
모니터 사진도 똑같아요. 사진은 아주 작은 점(픽셀) 수백만 개이고, 점마다 **밝기를 숫자로** 저장해요.

> 이번 프로그램은 3×3부터 **25×25(625칸)**까지 다뤄요.

필터 = 찾고 싶은 모양의 '정답 카드'



십자가 필터



X 필터

숨은그림찾기 할 때 손에 든 **정답 카드**처럼, 컴퓨터는 이 카드를 입력 그림 위에 **겹쳐 봐요.**

> 실제 data.json 필터에는 0.9, 0.3, 7.5 같은 숫자도 있어요. 숫자가 클수록 **그 칸이 더 중요하다**는 뜻이에요. 이 숫자를 **가중치**라고 불러요.

개념 3 · MAC 연산

MAC = 같은 칸끼리 곱하고, 모두 더하기

십자가 그림
× 십자가 필터



→ **점수 5**

십자가 그림
× X 필터



→ **점수 1**

MAC은 Multiply(곱하기)와 Accumulate(쌓아서 더하기)를 합친 말이에요. 같은 칸끼리 곱하면 겹치는 칸만 남기 때문에, 더한 점수가 클수록 그림과 필터가 많이 닮았다는 뜻이에요.

중요 · 과제 목표 ②

개념 4 · 판정

점수가 더 높은 필터가 이겨요

십자가 필터

5

승리!

VS

X 필터

1

아쉽게 패배

결과는 **십자가**!

그런데 점수가 **똑같으면?**
→ **판정 불가**
(영어로 UNDECIDED, 무승부)

필터마다 MAC 점수를 구해서 가장 높은 쪽을 정답으로 골라요. 두 점수가 사실상 같으면 억지로 고르지 않고 '판정 불가(UNDECIDED)'라고 정직하게 말해요.

개념 5 · 왜 NPU가 필요할까?

AI는 이 계산을 정말 정말 많이 해요

9번

3×3 필터 한 번

625번

25×25 필터 한 번

수억 번

진짜 AI · 사진 한 장

CPU = 선생님 1명

시험지를 **한 장씩 차례로** 채점해요. 똑똑하지만 1만 장이면 오래 걸려요. (직렬)

NPU = 조교 1,000명

O, X만 보는 단순한 일을 **동시에** 해요. 그래서 훨씬 빨라요. (병렬)

AI는 사진 한 장에 이 곱하기·더하기를 수억 번 해요. 그래서 하나씩 처리하는 CPU 대신, 수천 개를 동시에 처리하는 NPU라는 전용 칩이 필요하고, 요즘은 스마트폰에도 들어 있어요.

코드 0 · 프로그램 지도

main.py 한 파일, 479줄로 만들었어요

`1 · 직접 입력 (3×3)` `2 · data.json 채점` `3 · 보너스 속도 비교` `0 · 종료`

가장 중요한 **2번 메뉴**는 이렇게 흘러가요.

① **파일 읽기**
data.json 열기

→

② **필터 준비**
크기별 정답 카드

→

③ **문제 검사**
이름·크기·정답 확인

→

④ **채점**
MAC → 판정 → PASS/FAIL

→

⑤ **성적표**
속도표 + 통과/실패 수

약속: NumPy 같은 계산 도구 없이 **파이썬 기본 기능**(json, time, pathlib)만 썼어요.

코드 작성은 AI Claude의 도움을 받았어요. 코드를 직접 실행해 결과를 확인하고, 한 줄씩 이해한 내용을 이 발표에 담았어요.

코드 1 · 프로그램 시작과 메뉴

프로그램은 맨 아래에서 시작해 메뉴를 계속 돌아요

main.py 451~472줄 · main (메뉴 글자 출력 6줄은 생략)

```
def main():
    print("═══ Mini NPU Simulator ═══")
1   while True:
2       choice = input("선택: ").strip()
3       if choice == "1":
            mode_user_input()
        elif choice == "2":
            mode_json_analysis()
        elif choice == "3":
            mode_bonus_compare()
        elif choice == "0":
            print("프로그램을 종료합니다.")
4           break
        else:
            print("0, 1, 2, 3 중에서 선택하세요.")
```

⑤ main.py 475~479줄 · 시작점

```
if __name__ == "__main__":
    try:
        main()
    except (KeyboardInterrupt, EOFError):
        print("\n프로그램을 종료합니다.")
```

① while True · ④ break

0을 고를 때까지 반복하고, break에서 멈춰요.

② strip() · ③ if/elif/else

" 2 "도 2로 인식해서 맞는 모드로 보내요.

⑤ 왜 이렇게 시작할까?

직접 실행할 때만 메뉴를 열고, Ctrl+C에도 조용히 종료

파이썬에선 '리스트 안에 리스트'로 표를 만들어요

main.py 28~34줄 · create_matrix

```
def create_matrix(n, value=0.0):
    matrix = []
    for _ in range(n):
        row = [value] * n  # 한 줄(행)에 n개의 값
        matrix.append(row)
    return matrix
```

이렇게 생긴 표가 돼요

```
pattern = [[0, 1, 0],
           [1, 1, 1],
           [0, 1, 0]]
pattern[1][2]   # → 1 (1번 줄의 2번 칸)
```

한 줄씩 읽기

빈 표를 만들고 → n번 반복하면서 → 값이 n개인 줄을 만들어 → 표에 붙여요.

왜 n을 받을까?

크기를 정해 두지 않아서 **3×3도 25×25도 같은 코드**로 만들 수 있어요.

왜 0부터 셀까?

파이썬은 "첫 칸에서 몇 칸 떨어졌나"로 세요. 그래서 맨 앞이 **0번**이에요.

코드 3 · MAC 함수 (프로그램의 심장)

곱하고 더하기는 딱 7줄이에요

main.py 133~140줄 · mac (설명 줄은 생략)

```
def mac(pattern, filter_matrix):
1   n = len(pattern)
2   total = 0.0
3   for i in range(n):              # 행
        for j in range(n):          # 열
4           total += pattern[i][j] * filter_matrix[i][j]
5   return total
```

① 그림의 크기 N 알아내기

② 점수 통을 0으로 비우기

③ 모든 칸을 한 칸씩 방문 (**N×N번**)

④ 같은 칸끼리 곱해서 통에 더하기

왜 NumPy를 안 썼을까? 과제 규칙이에요. 도구가 대신 계산하면 원리가 안 보이니까, 반복문으로 **한 칸씩 직접** 계산했어요.

사람은 실수하니까, 잘못 쓴 줄만 다시 받아요

main.py 217~233줄 · input_matrix (안내 문구는 줄임)

```
def input_matrix(title, n=3):
    matrix = []
    while len(matrix) < n:
1       parts = input().split()
2       if len(parts) ≠ n:
            print("입력 형식 오류: …")
            continue
3       try:
            row = [float(p) for p in parts]
        except ValueError:
            print("입력 형식 오류: 숫자만 …")
            continue
        matrix.append(row)
    return matrix
```

① 공백으로 자르기

"0 1 0" → ['0', '1', '0']

② 개수 검사

"0 1"은 2개뿐 → 그 줄만 다시

③ 숫자 검사

"0 a 0" → a는 숫자로 못 바꿈 → 다시

왜?

오타 하나에 프로그램이 꺼지면 처음부터 다시 해야 해요.
continue = "아래는 건너뛰고 다시 처음으로"

중요 · 과제 목표 ③

코드 5 · 문제집 읽기 (모드 2)

문제 이름표에 그림 크기가 숨어 있어요

main.py 279~280줄 · load_json 중 일부

```
with open(path, "r", encoding="utf-8") as f:
    return json.load(f)
```

main.py 288~293줄 · get_size_from_key (설명 줄은 생략)

```
def get_size_from_key(key):
    parts = key.split("_")
    if len(parts) < 2 or parts[0] ≠ "size" or not parts[1].isdigit():
        return None
    return int(parts[1])
```

문제 이름 size_13_2의 가운데 13이 그림 크기예요. 이 숫자로 size_13 필터를 골라 채점하고, 이름이 규칙에 맞지 않으면 그 문제만 FAIL로 처리해요.

"size_13_2"를 넣으면



→ 13을 꺼내서 **size_13 필터**로 채점

utf-8
한글이 깨지지 않게 읽기

with
다 읽으면 파일을 자동으로 닫기

isdigit()
"size_ab"처럼 숫자가 아니면 걸러내기

중요 · 과제 목표 ② ③

코드 6 · 문제 하나가 채점되는 과정

문제 그림을 필터 2장과 비교하고, 정답과 맞춰 봐요

data.json 구조 (숫자는 줄임)

```
"filters": {                    ← 정답 카드
  "size_5": {
    "cross": [[...]],
    "x": [[...]] } },
"patterns": {
  "size_5_2": {
    "input": [[...]],           ← 문제 그림
    "expected": "+" } }         ← 채점용 정답
```

main.py 375~390줄 · analyze_pattern ⑥ 채점 (출력 줄 일부와 앞 들여쓰기 생략)

```
scores = {}
for label in STANDARD_LABELS:
    scores[label] = mac(pattern, filters[n][label])
verdict = judge(scores)
if verdict == expected:
    print(...)      # … | PASS
    return None     # 실패 이유 없음
# 다르면 FAIL과 이유를 돌려줌
```

① **size_5_2**
size_5 필터 고르기

→

② **Cross 필터**
MAC → **8.9점**

→

③ **X 필터**
MAC → **0.1점**

→

④ **판정**
높은 쪽 **Cross**

→

⑤ **정답과 비교**
“+” = Cross
→ **PASS**

코드 7 · 이름표 통일 (라벨 정규화)

‘+’와 ‘Cross’는 컴퓨터에겐 다른 글자예요

```
main.py 113~127줄 (설명 줄은 생략)
STANDARD_LABELS = ["Cross", "X"]

LABEL_ALIASES = {
    "+": "Cross",
    "cross": "Cross",
    "x": "X",
}

def normalize_label(label):
    text = str(label).strip().lower()    # 앞뒤 공백 제거 + 소문자로 통일
    return LABEL_ALIASES.get(text)       # 없으면 None
```

들어온 값	다듬기	결과
" + "	"+"	Cross
"CROSS"	"cross"	Cross
"x"	"x"	X
"?"	"?"	None → FAIL

data.json에는 같은 십자가가 ‘+’와 ‘cross’로 섞여 있어요. 컴퓨터에겐 다른 글자라서, 읽자마자 표에서 찾아 Cross와 X로 통일해야 정답과 바르게 비교할 수 있어요.

코드 8 · 크기와 숫자 검사

계산하기 전에 “제대로 된 N×N 숫자 표”인지 확인해요

main.py 47~58줄 · is_valid_matrix (설명 줄은 생략)

```
def is_valid_matrix(matrix, n):
1   if not isinstance(matrix, list) or len(matrix) ≠ n:
        return False
    for row in matrix:
2       if not isinstance(row, list) or len(row) ≠ n:
            return False
        for value in row:
            # True/False도 파이썬에서는 숫자로 취급되므로 따로 제외한다
3           if isinstance(value, bool) or not isinstance(value, (int, float)):
                return False
    return True
```

① 줄 수
리스트인지, 줄이 N개인지

② 칸 수
줄마다 칸이 N개인지

③ 숫자
칸마다 진짜 숫자인지

isinstance(값, 종류)
종류가 맞는지 확인

왜 True를 따로 빼요?

파이썬에선 **True + True → 2**처럼 True가 1로 계산돼요. 데이터에 실수로 true가 있어도 숫자로 통과하지 않게 막아요.

코드 9 · 필터 준비

필터는 이름표를 통일하고, 다 모였을 때만 써요

main.py 310~318줄 · ① 이름표 바꾸고 검사 (앞 들여쓰기 생략)

```
loaded = {}
for label_key, matrix in group.items():
    label = normalize_label(label_key)
    if label is None:
        print(...)  # 알 수 없는 필터 이름
    elif not is_valid_matrix(matrix, n):
        print(...)  # 필터 크기 오류
    else:
        loaded[label] = matrix
```

main.py 320~330줄 · ② 빠진 필터 찾기 (일부 생략)

```
missing = []
for label in STANDARD_LABELS:
    if label not in loaded:
        missing.append(label)

if not missing:
    filters[n] = loaded  # ✓ 로드 완료
else:
    print(...)  # ✗ 없는 필터: T
```

필터 이름도 통일

data.json의 “cross”, “x”를 Cross, X로 바꿔서 저장해요.

틀린 필터는 버리기

크기가 안 맞는 필터는 쓰지 않고 이유만 출력해요.

빠진 게 있으면 그 크기는 안 써요

STANDARD_LABELS가 다 있어야 해요. 그래서 T를 추가하면 **T 필터도 필수**가 돼요.

코드 10 · 판정 함수

1등을 찾고, 1등과 거의 같은 점수가 있는지 확인해요

main.py 163~179줄 · judge (설명 줄은 생략)

```python
def judge(scores):
    # ① 가장 높은 점수 찾기
    best_label = None
    best_score = None
    for label, score in scores.items():
        if best_score is None or score > best_score:
            best_label = label
            best_score = score

    # ② 1등과 거의 같은 점수가 또 있으면 판정 불가
    for label, score in scores.items():
        if label ≠ best_label and abs(best_score - score) < EPSILON:
            return "UNDECIDED"
    return best_label
```

① 키 재기

한 명씩 보면서 “지금까지 제일 큰 사람”보다 크면 바꿔요. 끝나면 남은 사람이 1등.

② 무승부 검사

1등과 차이가 **0.000000001보다 작은** 라벨이 있으면 UNDECIDED.

점수 상자 · items()

{라벨: 점수}에서 items()가 (라벨, 점수)를 한 쌍씩 꺼내요. 라벨이 **3개, 4개로 늘어도** 그대로 써요.

6번 검사하고, 문제가 있으면 그 자리에서 이유를 돌려줘요

main.py 357~366줄 · analyze_pattern 중 ④ 검사 (6개 검사가 모두 이 모양)

```
    # ④ 패턴 크기/값 검사
    pattern = case["input"]
    if not is_valid_matrix(pattern, n):
        actual = describe_size(pattern)
        if actual == f"{n}×{n}":
            reason = "값 오류 (숫자가 아닌 값이 있음)"
        else:
            reason = f"크기 불일치 (필터 {n}×{n}, 패턴 {actual})"
        print(f"FAIL: {reason}")
        return reason
```

return을 만나면 함수가 그 자리에서 끝나요. 그래서 문제가 있으면 다음 검사로 안 넘어가고, 모두 통과해 PASS면 **None(없음)**을 돌려줘요.

① 문제 이름	size_N_번호 모양인가
② input · expected	그림과 정답이 다 있나
③ 필터	size_N 필터가 준비됐나
④ 크기·숫자	N×N 숫자 표인가 ← 왼쪽 코드
⑤ 정답 이름표	+, x처럼 아는 이름인가
⑥ 채점	MAC → 판정 → 정답과 비교

실패 이유만 모아 두면 성적표가 완성돼요

main.py 407~424줄 · mode_json_analysis 중 일부

```
failures = []          # (케이스 이름, 실패 사유) 목록
for key, case in patterns.items():
    reason = analyze_pattern(key, case, filters)
    if reason is not None:
        failures.append((key, reason))
# … (성능 분석 표 출력)
total = len(patterns)
print(f"총 테스트: {total}개")
print(f"통과: {total - len(failures)}개")
print(f"실패: {len(failures)}개")
if failures:
    print("실패 케이스:")
    for key, reason in failures:
        print(f"- {key}: {reason}")
```

실제 출력

```
총 테스트: 6개
통과: 3개
실패: 3개
실패 케이스:
- size_5_1: 동점(UNDECIDED)…
- size_13_2: 동점(UNDECIDED)…
- size_25_1: 동점(UNDECIDED)…
```

통과 수는 따로 안 세요

전체 − 실패 = 통과. 셀 게 하나라 안 어긋나요.

(이름, 이유) 한 쌍으로 저장

어느 문제가 왜 틀렸는지 한눈에 보여요.

오차 1 · 부동소수점

컴퓨터도 계산 실수를 한다고?!

파이썬에서 직접 해 보기

```
>>> 0.1 + 0.2
0.30000000000000004
>>> total = 0.0
>>> for _ in range(9): total += 0.1   # mac()처럼 하나씩 더하기
>>> total
0.8999999999999999   # 0.9가 아니에요!
```

컴퓨터는 0.1을 2진수로 정확히 저장하지 못해 아주 작은 오차가 생기고, 여러 번 더하면 그 오차가 쌓여요. 그래서 수학적으로 같은 점수도 끝자리가 달라질 수 있어요.

※ 파이썬 3.12부터 sum()은 오차를 보정해서 0.9가 나와요. 우리 mac()처럼 += 로 하나씩 더할 때 오차가 보여요.

오차 2 · 해결책 epsilon(엡실론) 규칙

차이가 아주아주 작으면 ‘무승부’로 정해요

```
abs(A - B) < 0.000000001 → 무승부
```

저울 두 개가 1.000kg과 0.999999kg을 가리키면 ‘같은 무게’라고 보는 게 맞죠? 그 ‘이 정도는 같다’는 기준이 **epsilon**이에요. 코드에서는 **EPSILON = 1e-9**(10억분의 1)로 정했어요.

두 점수의 차이가 10억분의 1보다 작으면 같은 점수로 봐요. 계산 오차는 이보다 훨씬 작고 진짜 점수 차이는 훨씬 크니까, 오차 때문에 승자가 바뀌는 일을 막을 수 있어요.

오차 3 · 실제 data.json 채점 결과

6문제 중 3개 통과, 3개는 무승부

문제	Cross 점수	X 점수	판정	결과
size_5_1	0.9	0.8999999999999999	UNDECIDED	FAIL
size_5_2	8.9	0.1	Cross	PASS
size_13_1	0.3	14.700000000000008	X	PASS
size_13_2	7.499999999999997	7.5	UNDECIDED	FAIL
size_25_1	4.9	4.899999999999999	UNDECIDED	FAIL
size_25_2	52.9	0.1	Cross	PASS

> UNDECIDED = 무승부. 실패 3개는 **버그가 아니에요!** 수학으로는 똑같은 점수라서 무승부 규칙이 **제대로 작동한 결과**예요. (무승부는 정답과 다르니 과제 규칙상 FAIL)

오차 4 · 직접 검산하기

손으로 계산하면 세 문제 모두 정확히 같은 점수예요

문제	수학 계산 (손으로)	컴퓨터 결과 (Cross vs X)	규칙이 없었다면
size_5_1 X 그림	Cross 0.9×1칸 = **0.9** X 0.1×9칸 = **0.9**	`0.9` vs `0.8999999999999999`	**Cross 승 (틀림)**
size_13_2 십자가 그림	Cross 0.3×25칸 = **7.5** X 7.5×1칸 = **7.5**	`7.499999999999997` vs `7.5`	**X 승 (틀림)**
size_25_1 X 그림	Cross 4.9×1칸 = **4.9** X 0.1×49칸 = **4.9**	`4.9` vs `4.899999999999999`	**Cross 승 (틀림)**

세 문제 모두 손으로 계산하면 두 점수가 정확히 같아요. 작은 수를 여러 번 더한 쪽에만 오차가 쌓였기 때문에, epsilon 규칙이 없으면 실력이 아니라 계산 오차가 승자를 정하게 돼요.

속도 1 · 시간 재는 법

계산 시간은 스톱워치로 10번 재서 평균을 내요

main.py 185~192줄 · measure_average_ms (첫 줄은 화면에 맞게 두 줄로, 설명 줄은 생략)

```
def measure_average_ms(mac_function, pattern,
                       filter_matrix, repeat=REPEAT_COUNT):
    start = time.perf_counter()
    for _ in range(repeat):
        mac_function(pattern, filter_matrix)
    end = time.perf_counter()
    return (end - start) * 1000 / repeat     # 초 → 밀리초, 그리고 평균
```

perf_counter

아주 정밀한 스톱워치

왜 10번?

한 번은 너무 빨라 흔들려요

계산만 재기

출력·파일 읽기는 빼요

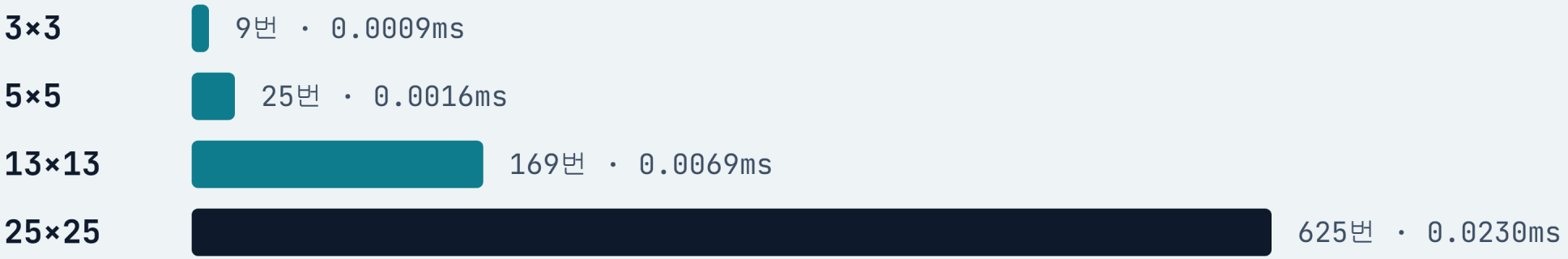
× 1000

초 → 밀리초 (1초 = 1000ms)

계산하는 부분만 스톱워치로 10번 재서 평균을 냈어요. 한 번은 너무 빨라 값이 흔들리고, 화면 출력이나 파일 읽기는 훨씬 느려서 빼야 계산 시간만 정확히 잴 수 있어요.

속도 2 · 성능 분석 결과

그림이 커지면 계산은 얼마나 늘까?



N×N칸을 하나씩 다 계산하니까 **계산 횟수 = N × N**. 5×5 → 25×25면 계산 **25배**, 시간도 약 **14배**.

한 변이 **2배**면 계산은 **4배** → 이걸 **O(N²)**, "오 엔 제곱"이라고 불러요.

MAC은 N×N칸을 한 번씩 계산하니 계산 횟수가 N^2이에요. 5×5에서 25×25로 계산이 25배 늘 때 시간도 약 14배 늘어서, 시간이 N^2에 거의 비례한다는 근거가 돼요.

중요 · 과제 목표 ⑥

고장 찾기 · 실패 원인 분석

FAIL이 나오면 탐정처럼 3곳을 살펴봐요

① 문제지가 이상해?

이름 규칙, 그림 크기, 정답 표시가 약속대로인지 (데이터)

이번엔 0건

② 계산법이 틀렸어?

곱하기·더하기·판정 코드가 맞는지 (로직)

이번엔 0건

③ 비교 방법이 문제?

계산 오차 때문에 무승부가 됐는지 (수치 비교)

이번엔 3건

FAIL이 나오면 데이터 → 로직 → 수치 비교 순서로 확인해요. 무승부가 아닌 문제가 모두 정답이라 로직은 정상이었고, 실패 이유를 글로 남겨 두니 고친 뒤 다시 채점해 개선을 확인할 수 있어요.

새 모양 ‘T’를 알아보게 하려면? 6단계

1 · 이름표 추가

main.py 표에 **"T"**와 **"t": "T"** 한 줄씩

2 · 이름표 검사

" T ", "t"를 넣어 T가 나오는지, 기존 "+"·"x"는 그대로인지

3 · 판정·출력 확인

코드 수정 없음! 표를 따라 **T 점수**가 자동으로 계산·출력돼요

4 · 필터 추가

data.json에 T 필터를 **모든 크기**(5, 13, 25)에 넣기

5 · 문제 추가

정답이 T인 문제를 만들어 PASS가 나오는지 확인

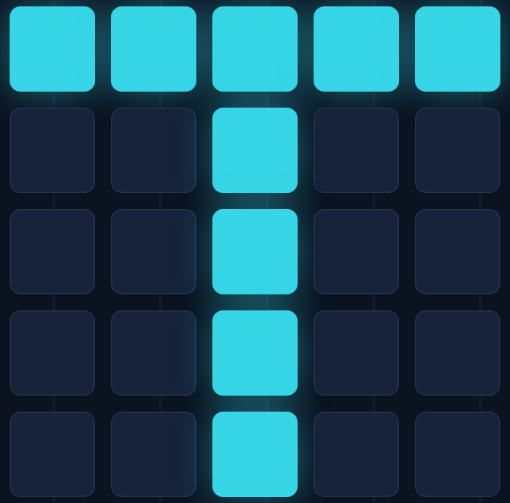
6 · 전체 다시 채점

새 문제뿐 아니라 **기존 문제 결과가 그대로인지** 비교

이미 있는 모양의 새 이름(예: **plus** → Cross)이라면 **1 · 2 · 5 · 6단계**만 하면 돼요. 이 절차는 평가 피드백을 받고 README에 정리했어요.

확장 2 · 직접 해 본 실험 (복사본에서)

T를 넣어 보니, 함정이 2개 있었어요



```
STANDARD_LABELS = [..., "T"]
"t": "T",  # 이름표 표에 추가
```

성공 · T 그림, 정답 T

Cross 4.9 · X 0.3 · **T 9.0** → **T · PASS**

함정 1 · size_5에만 T 필터를 넣음

"size_13 필터 로드 실패 (없는 필터: T)" → 13×13, 25×25 문제 **4개가 전부 FAIL**

함정 2 · 원래 있던 X 문제(size_5_1)

Cross 0.9 · X 0.8999… · **T 3.0** → 무승부였던 결과가 **T로 뒤집힘**

교훈: 새 기능이 잘 되는지만 보면 안 되고, **원래 되던 것이 그대로인지**도 다시 확인해야 해요. 이걸 **회귀 테스트**라고 해요.

보너스 2가지도 만들었어요

① 한 줄로 펼치기

0	1	0	1	1	1	0	1	0

표를 **한 줄 리스트**로 펴면 pattern[i][j](두 번 찾기)가 pattern[k](한 번 찾기)로 줄어요. README 측정에서 **약 15~38% 빨랐어요**(잴 때마다 달라요).

하지만 계산 횟수는 그대로 N^2이라 **O(N^2)는 같아요.**

② 모양 자동 생성기

```
make_cross(25)  make_x(25)
```

크기 N만 넣으면 그림을 만들어 줘요.
십자가 = 가운데 가로줄 + 세로줄이 1
X = 두 대각선이 1

속도 측정용 그림을 25×25까지 **손으로 안 쳐도** 돼요.

보너스 코드 · 모양 생성기와 한 줄 펼치기

보너스 기능도 코드로 보면 간단해요

main.py 90~106줄 · make_cross, make_x (설명·주석 생략)

```
def make_cross(n):
    matrix = create_matrix(n)
1   center = n // 2
    for i in range(n):
        set_value(matrix, center, i, 1.0)
        set_value(matrix, i, center, 1.0)
    return matrix

def make_x(n):
    matrix = create_matrix(n)
    for i in range(n):
        set_value(matrix, i, i, 1.0)
2       set_value(matrix, i, n - 1 - i, 1.0)
    return matrix
```

main.py 143~157줄 · flatten, mac_flat (설명 줄은 생략)

```
def flatten(matrix):
    flat = []
    for row in matrix:
        for value in row:
            flat.append(value)
    return flat
def mac_flat(flat_pattern, flat_filter):
    total = 0.0
3   for k in range(len(flat_pattern)):
        total += flat_pattern[k] * flat_filter[k]
    return total
```

① **n // 2** = 나눈 몫 → 25 // 2 = 12번이 가운데 · ② **n − 1 − i** = 반대쪽 대각선 칸 · ③ 반복문이 1개라 칸을 한 번에 찾아요

깜짝 퀴즈 · 클릭하면 정답

오늘 배운 것, 3문제로 확인해 봐요

Q1

X 그림에 십자가 필터를 겹치면 점수는?
(3×3, 0과 1만)

1점 · 가운데 한 칸만 겹쳐요

Q2

50×50 그림이면 곱하기·더하기는 몇 번?

2,500번 · $50 \times 50 = N^2$

Q3

파이썬에서 0.1 + 0.2 == 0.3 의 결과는?

False(거짓) ·
0.30000000000000004라서

보너스 질문: 그럼 두 점수가 같은지는 어떻게 비교해야 할까? → **abs(A − B) < 1e-9**

중요 · 과제 목표 정리

과제 목표 정리 1/2 · 원리

과제 목표 ①~③을 이렇게 정리했어요

① **MAC 연산이 무엇이고 AI에서 왜 중요한가**

MAC은 같은 위치의 숫자끼리 **곱하고(Multiply)** 그 결과를 모두 **더하는(Accumulate)** 계산이에요. AI는 이미지를 판별할 때 이 계산을 사진 한 장에 수억 번 반복하기 때문에, 많은 MAC을 **동시에 처리하는 NPU** 같은 전용 칩이 필요해요.

근거 · 5, 7, 11장

② **곱하고 더해서 유사도(점수)를 계산하는 원리**

필터는 찾고 싶은 모양에서 중요한 칸에 큰 값(**가중치**)을 둔 표예요. 0을 곱하면 0이 되므로 그림과 필터가 **겹치는 칸만 점수에 남고**, 점수가 클수록 더 닮았다는 뜻이에요. 십자가 그림은 십자가 필터와 5점, X 필터와 1점이라 십자가로 판정해요.

근거 · 5, 6, 14, 18장

③ **data.json 키·라벨 규칙과 정규화하는 이유**

문제 이름 **size_13_2**에서 13이 그림 크기라서 size_13 필터로 채점해요. 정답은 '+', 'x', 필터 키는 'cross', 'x'처럼 표기가 섞여 있어서, 읽자마자 표(LABEL_ALIASES)로 **Cross·X로 통일**해야 판정과 정답을 같은 기준으로 비교할 수 있어요.

근거 · 13, 14, 15, 17장

중요 · 과제 목표 정리

과제 목표 ④~⑥을 이렇게 정리했어요

④

부동소수점 오차와 epsilon 비교가 필요한 이유

컴퓨터는 0.1 같은 소수를 2진수로 정확히 저장하지 못해서, 여러 번 더하면 **0.8999999999999999**처럼 오차가 쌓여요. 수학적으로 같은 점수도 끝자리가 달라지므로, 차이가 **1e-9보다 작으면 동점(UNDECIDED)**으로 봐요. 실제 data.json의 실패 3건이 모두 이 경우였어요.

근거 · 21, 22, 23, 24장

⑤

크기별 시간 측정과 O(N^2)의 근거

계산하는 부분만 perf_counter로 **10번 재서 평균**을 냈어요. MAC은 N×N칸을 한 번씩 계산하니 횟수가 N^2이고, 5×5 → 25×25에서 계산은 25배, 시간은 약 14배 늘어 **N^2에 거의 비례**했어요. 작은 크기는 함수 호출 같은 고정 비용 때문에 정확히 비례하지 않아요.

근거 · 25, 26장

⑥

실패 원인을 나눠 진단하고 개선하기

데이터/스키마(이름·크기·정답 표기) → **로직**(계산 코드) → **수치 비교**(오차) 순서로 확인해요. 이번엔 0건 / 0건(동점 아닌 3문제 모두 정답) / 3건이었어요. 실패 이유를 글로 남겨 원인을 바로 찾고, 고친 뒤 **전체를 다시 채점**해 개선을 확인해요.

근거 · 19, 27, 29장

한 문장 정리

AI가 '본다'는 건 숫자끼리 곱하고 더하는 것이에요

그리고 좋은 프로그램은 **오차를 알고, 실수에 멈추지 않고, 넓히기 쉬운** 프로그램이에요.

